IT5002 Computer Systems and Applications
Tutorial 9 Suggested Solutions

Question 1

Consider the following segment table:

Segment	Base	Length
0	219	600
1	2300	14
2	1327	580
3	1952	96

What are the physical addresses for the following logical addresses? Which of these addresses, if accessed, would result in a segmentation violation?

(a) (0,430)

219+430, legal access

(b) (1,10)

2300+10, legal access

(c) (2,500)

1327+500, legal access

(d) (3,400)

1952+400, illegal access

Question 2

Assume there is a 1,024KB segment where memory is allocated using the buddy system. Describe the configuration of the system as the following allocation requests are served:

(a) Request 240 bytes

256 allocated, 256 free, 512 free

(b) Request 120 bytes

256 allocated, 128 allocated, 128 free, 512 free

(c) Request 60 bytes

256 allocated, 128 allocated, 64 allocated, 64 free, 512 free

(d) Request 130 bytes

256 allocated, 128 allocated, 64 allocated, 64 free, 256 allocated, 256 free

Next, explain the configuration of the system as the blocks allocated first, third, and fourth are released.

(e) -- first block freed
256 f, 128 a, 64 a, 64 f, 256 a, 256 f

(f) -- third block freed
256 f, 128 a, 128 f, 256 a, 256 f

(g) -- fourth block freed
256 128 a, 128 f, 512 f

Question 3.

This question is based on a "simple" file system built from the various components discussed in the lecture.

Partition Information (Free Space Information)

- Bitmap is used, with a total of 32 file data blocks (1 = Free, 0 = Occupied):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	1	0	0	1	0	0	0	0	1	0	1
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**
1	0	1	0	1	0	1	0	1	1	1	0	0	1	0	0

Directory Structure + File Information:

- Directory structures are stored in 4 "directory" blocks. Directory entries (both files and subdirectories) of a directory are stored in a single directory block.
- Directory entry:
  - **For File:** Indicates the first and last data block number.
  - **For Subdirectory:** Indicates the directory structure block number that contains the subdirectory's directory entries.
  - The "**/**" root directory has the directory block number 0.

0	1	2	3
y |Dir | 3 f |File| 12| 2 x |Dir | 1	g |File| 0| 31 z |Dir | 2	k |File| 6| 6	i |File| 1| 3 h |File|27|28

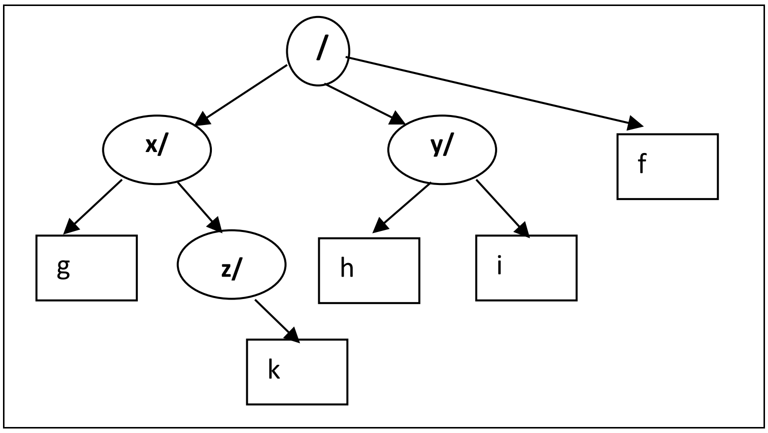
File Data:

- Linked list allocation is used. The first value in the data block is the "next" block pointer, with "-1" to indicate the end of data block.
- Each data block is 1 KB. For simplicity, we show only a couple of letters/numbers in each block.

0	1	2	3	4	5	6	7
11 AL	9 TH	-1 S!	-1 ND	23 GS	-1 SO	-1 :)	10 TE
8	**9**	**10**	**11**	**12**	**13**	**14**	**15**
31 RE	3 EE	28 M:	31 OH	19 SE	13 AH	4 IN	17 NO
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**
30 YE	2 OU	1 ON	17 RI	26 EV	14 AT	21 DA	7 YS
24	**25**	**26**	**27**	**28**	**29**	**30**	**31**
-1 HO	18 ME	0 AL	30 OP	-1 -(	5 LO	21 ER	-1 A!

a. (Basic Info) Give:
  - The current free capacity of the disk.
  - The current user view of the directory structure.

- **~12KB free space (12 '1' in Bitmap, each data block is 1KB). Due to the linked list file allocation overhead, the actual capacity is a little bit smaller.**



b. (File Paths) Walkthrough the file path checking for:
  - "`/y/i`"
  - "`/x/z/i`"

Only the directory block number is indicated:
- `/y/i`: 0, 3 (successful)
- `/x/z/i`: 0, 1, 2 (failed)

c. (File access) Access the entire content for the following files:
  - "`/x/z/k`"
  - "`/y/h`"

- `/x/z/k`: block 6, content = "`:)`"
- `/y/h`: blocks 27, 30, 21, 14, 4, 23, 7, 10, 28, content = "`OPERATINGSYSTEM:-(`"

d. (Create file) Add a new file "`/y/n`" with 5 blocks of content. You can assume we always use the free block with the smallest block number. Indicate **all changes required to add the file.**

Bitmap updated: Bit 5, 8, 13, 15, 16 changed to 0
Directory block 3 (for /y) updated: "n |File| 5|16" added
Data Blocks 5, 8, 13, 15, 16 (next block pointer changed, with -1 in block 16).

Question 4.

Consider the following virtual page references:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

How many page faults would occur for the following replacement algorithms, assuming three, four, and seven physical pages? Remember that physical pages are initially empty, so your first unique pages will cost one fault each.

(a) LRU replacement
(b) FIFO replacement

LRU 3 Frames

Time	Page 0	Page 1	Page 2
1	1*		
2	1	2*	
3	1	2	3*
4	4*	2	3
5	4	2	3
6	4	2	1*
7	5*	2	1
8	5	6*	1
9	5	6	2*
10	1*	6	2
11	1	6	2
12	1	3*	2
13	7*	3	2
14	7	3	6*
15	7	3	6
16	2*	3	6
17	2	3	1*
18	2	3	1
19	2	3	1
20	2	3	6*

Page faults: 15

LRU 4 Frames

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Time	Page 0	Page 1	Page 2	Page 3
1	1*			
2	1	2*		
3	1	2	3*	
4	1	2	3	4*
5	1	2	3	4

6	1	2	3	4
7	1	2	5*	4
8	1	2	5	6*
9	1	2	5	6
10	1	2	5	6
11	1	2	5	6
12	1	2	3*	6
13	1	2	3	7*
14	6*	2	3	7
15	6	2	3	7
16	6	2	3	7
17	6	2	3	1*
18	6	2	3	1
196	6	2	3	1
20	6	2	3	1

Page faults: 10

Solution for 7 physical pages omitted.

FCFS 3 Frames

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Time	Page 0	Page 1	Page 2
1	1*		
2	1	2*	
3	1	2	3*
4	4*	2	3
5	4	2	3
6	4	1*	3
7	4	1	5*
8	6*	1	5
9	6	2*	5
10	6	2	1*
11	6	2	1
12	3*	2	1
13	3	7*	1
14	3	7	6*
15	3	7	6
16	2*	7	6
17	2	1*	6
18	2	1	6
19	2	1	3*
20	6*	1	3

Number of page faults: 16

FCFS 4 Frames

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Time	Page 0	Page 1	Page 2	Page 3
1	1*			
2	1	2*		
3	1	2	3*	
4	1	2	3	4*
5	1	2	3	4
6	1	2	3	4
7	5*	2	3	4
8	5	6*	3	4
9	5	6	2*	4
10	5	6	2	1*
11	5	6	2	1
12	3*	6	2	1
13	3	7*	2	1
14	3	7	6*	1
15	3	7	6	1
16	3	7	6	2*
17	1*	7	6	2
18	1	7	6	2
19	1	3*	6	2
20	1	3	6	2

# of page faults: 14

Question 5.

In this question we will consider a virtual memory system with 64 bytes per page, 10 bit virtual addresses and 9 bit physical addresses.

a. What is the maximum size in bytes of the virtual memory? Physical memory?

   1024 bytes of VM, 512 bytes of physical memory.

b. What is the maximum number of virtual and physical pages?

   Byte index is 6 bits (2^6=64), leaving 4 bits for VPN or 16 virtual pages, and 3 bits for PPN or 8 physical pages.

c. Using the page table below, translate the following virtual addresses to physical addresses:

   0, 15, 576, 987, 1020

	V	PPN
0	1	2
1	1	3
2	0	(13,5,17)

3	0	(12,5,6)
4	1	4
5	1	7
6	0	(10,3,1)
7	0	(3,4,1)
8	0	(7,5,5)
9	1	6
10	0	(6,3,13)
11	0	(7,1,16)
12	1	5
13	0	(6,4,1)
14	0	(6,8,3)
15	1	1

Virtual Address	Binary (page number/Byte Index)	Virtual Page Number
0	0000 000000	0
15	0000 001111	0
576	1001 000000	9
987	1111 011011	15
1020	1111 111100	15

Using the table above we get the following translations:

Virtual Address	VPN	PPN	Binary (PPN/Byte Index)	Physical Address
0	0	2	010 000000	128
15	0	2	010 001111	143
576	9	6	110 000000	384
987	15	1	001 011011	91
1020	15	1	001 111100	124